UTN SANTA FE

Diagrama de clases, DAO y DTO

CÁTEDRA DE DISEÑO DE SISTEMAS — CLASE PRÁCTICA

UTN SANTA FE

Ejemplo: sistema de biblioteca

Tenemos una entidad Libro y necesitamos:

- Buscar por ISBN
- Guardar libros nuevos
- Actualizar stock
- Eliminar libros
- Mostrarlo en pantalla web
- Enviarlo a una UI Desktop
- Registrarlo en logs

⚠ **Pregunta:** ¿qué pasa si todo eso vive en la misma clase?

La tesis de la clase

> "Cada clase debería tener una sola razón para cambiar."

Si tu clase Libro cambia porque:

- Cambió el modelo en la base de datos, **Y ADEMÁS**
- Cambió el diseño de la pantalla,

...esa clase tiene **dos Responsabilidades**. Ninguna clase debería tener dos Responsabilidades.



UTN SANTA FE

Presentando al DTO

DTO = Data Transfer Object

Objeto de Transferencia de Datos.

Analogía: un DTO es una **valija**.

- No piensa.
- No decide.
- No tiene lógica.
- Solo transporta cosas de un lado a otro.

UTN SANTA FE

Cómo se ve un DTO en código

```
public class LibroDTO {
    private String isbn;
    private String titulo;
    private String autor;
    private int stockDisponible;

    // solo getters y setters
    // NO hay lógica de negocio
    // NO hay conexión a base de datos
}
```

Es casi aburrido. Ese es el punto.

UTN SANTA FE

¿Por qué necesito un DTO si ya tengo la entidad?

Imagina que la tabla libros tiene la columna precio_costo – información **interna, sensible**.

Si devolvés la entidad completa a tu UI:

Ese dato viaja al frontend.	Queda expuesto en el Navegador.	Cualquiera lo ve con el inspector del navegador.

El DTO es el filtro. Es la valija que armamos, a propósito, con **solo** lo que querés que viaje.

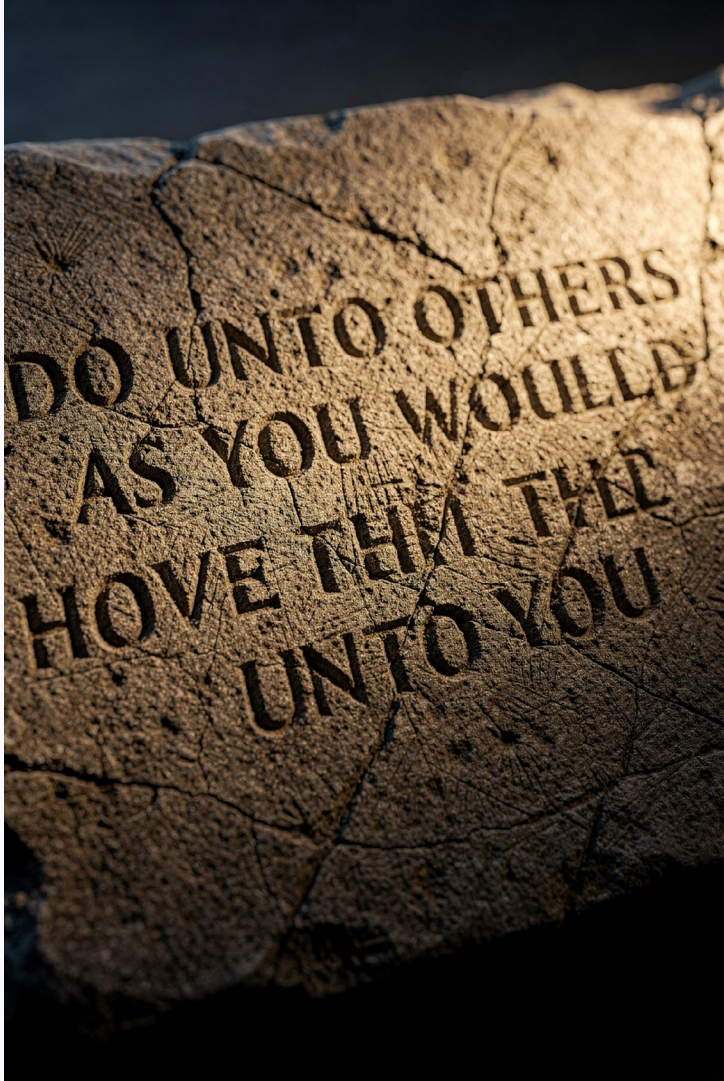
UTN SANTA FE

Regla de oro del DTO

Si un DTO tiene un método que **calcula**, **valida** o **llama a otra clase**...

Dejó de ser un DTO.

Se convirtió en otra cosa. Y probablemente mal diseñada.



UTN SANTA FE

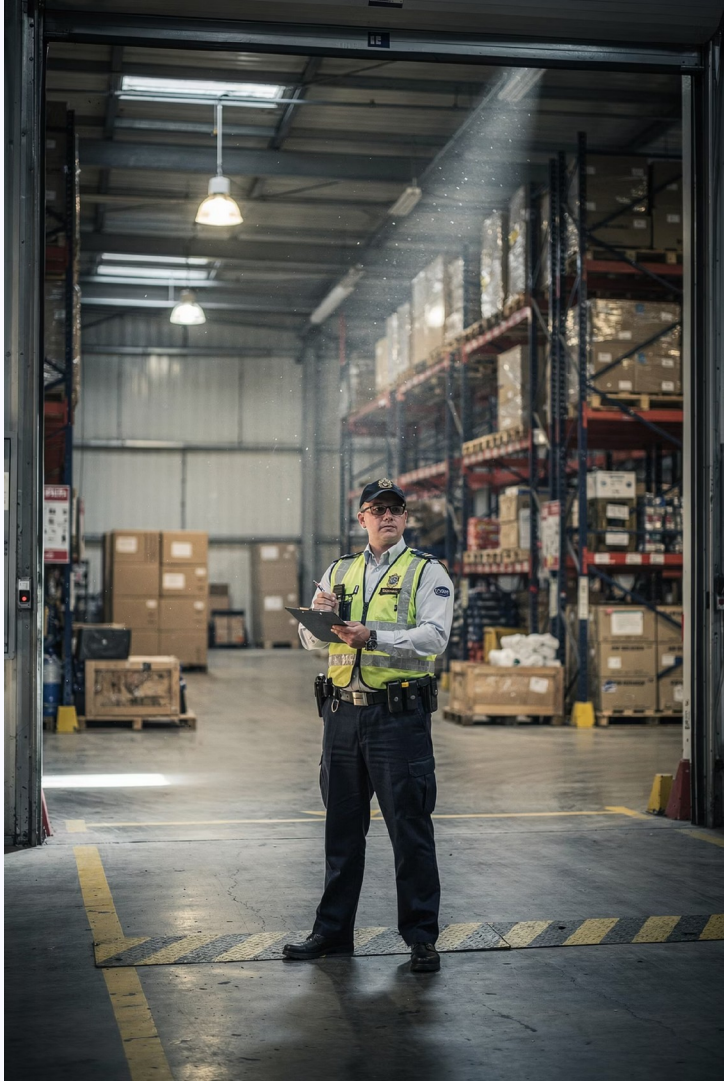
Presentando al DAO

DAO = Data Access Object

Objeto de Acceso a Datos.

Analogía: el DAO es el **empleado de un depósito**.

- Es el único autorizado a entrar al depósito (la base de datos).
- Nadie más habla directo con la base.
- Todo pasa por él.

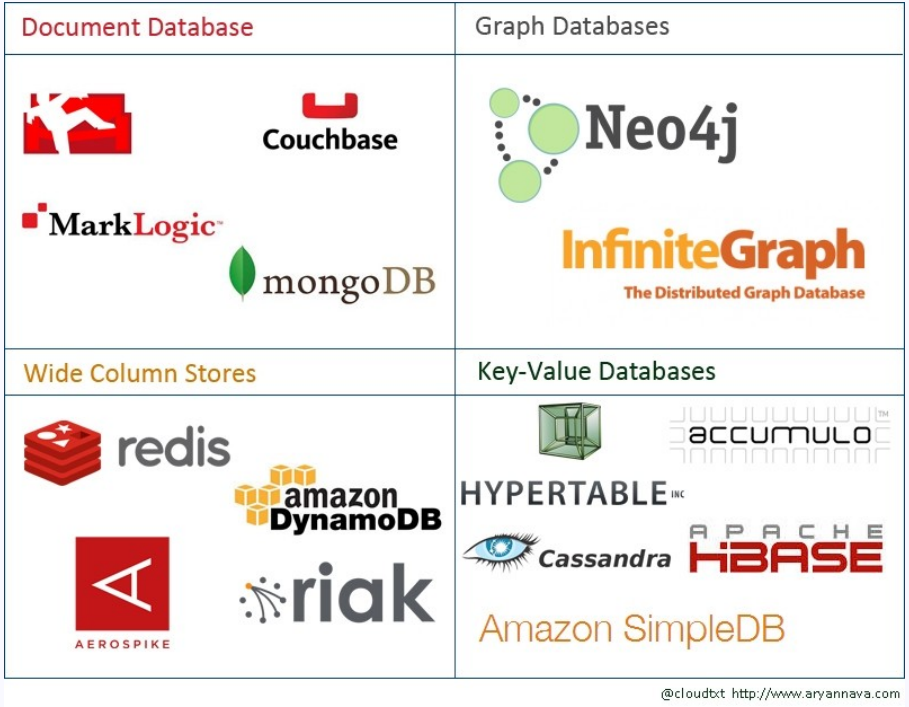


Patron DAO (Data Access Object)

- Existe una gran variedad de bases de datos disponibles → Esta variedad no debe afectar a la lógica de nuestro software

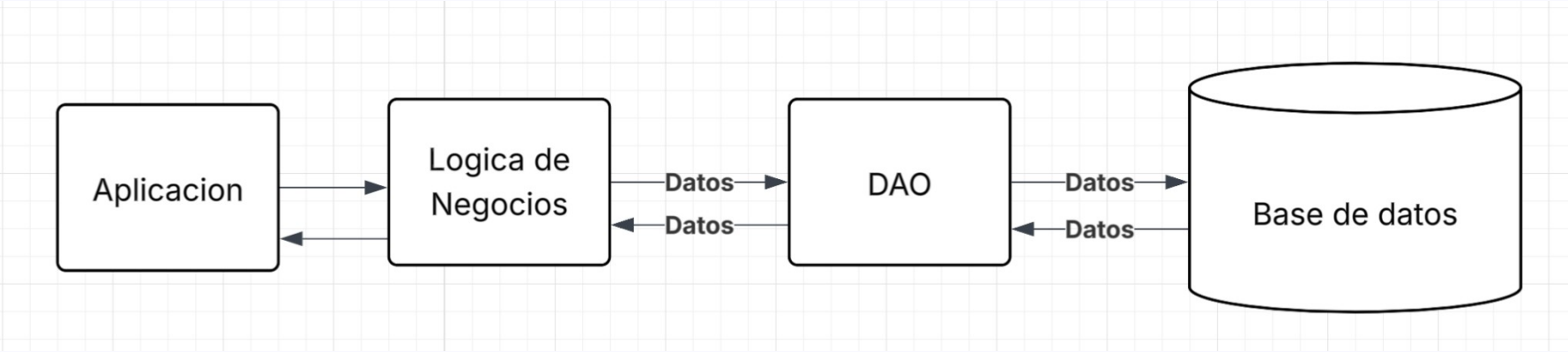
  Bajar acoplamiento

  Disminuir impacto en upgrades/cambios



DAO (Data Access Object)

- Se expone un objeto que encapsula la lógica necesaria para poder llevar los datos desde las clases del dominio del problema (capa de negocio) hacia la persistencia (capa de datos) y viceversa.



DAO (Data Access Object)

- Este objeto suele ofrecer métodos para manipular elementos:

  - Añadir
  - Actualizar
  - Buscar
  - Borrar

Cómo se ve un DAO en código

```
public interface LibroDAO {
    Libro buscarPorIsbn(String isbn);
    List listarTodos();
    void guardar(Libro libro);
    void actualizarStock(String isbn, int nuevoStock);
    void eliminar(String isbn);
}
```

Notá algo clave: es una **interfaz**, no una clase.

UTN SANTA FE

¿Por qué interfaz y no clase directa?

Porque con una interfaz puedo tener **múltiples implementaciones**:

```
public class LibroDAOMySQL implements LibroDAO {
    // consultas SQL acá
}

public class LibroDAOMongo implements LibroDAO {
    // consultas Mongo acá
}

public class LibroDAOFake implements LibroDAO {
    // datos en memoria, para tests
}
```

Y el resto del sistema **nunca se entera** cuál se está usando.

UTN SANTA FE

Esto que acabamos de hacer se llama:

Inversión de Dependencias

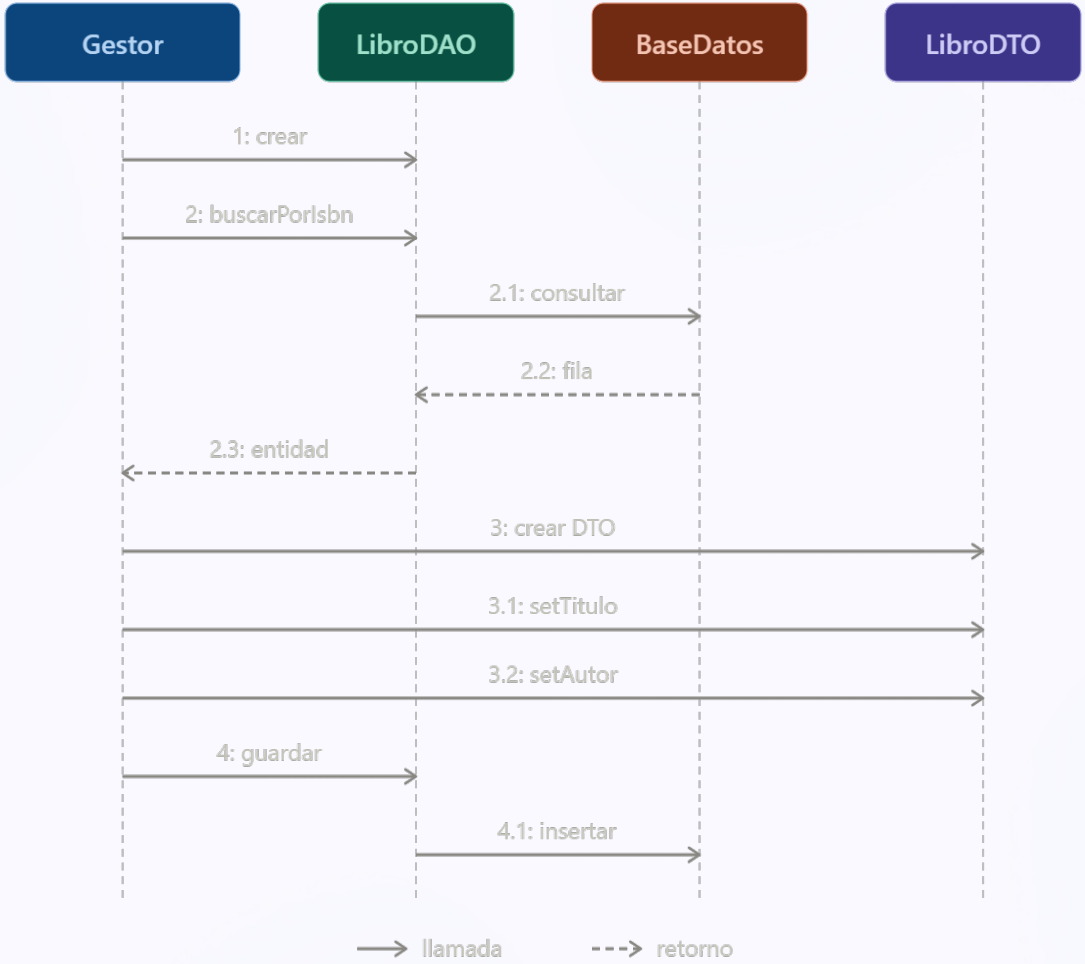
Mi lógica de negocio depende de una **abstracción** (la interfaz), no de una **implementación concreta**.

Consecuencias lógicas:

Cambio la base de datos sin tocar el resto del sistema.

Testeo la lógica de negocio sin necesitar una base real corriendo.

Puedo desarrollar en paralelo mientras el equipo de infraestructura decide qué motor usar.


Gestor
LibroDAO
BaseDatos
LibroDTO
1: crear
2: buscarPorIsbn
2.1: consultar
2.2: fila
2.3: entidad
3: crear DTO
3.1: setTitulo
3.2: setAutor
4: guardar
4.1: insertar
llamada
retorno

Errores combinando DAO y DTO

Miren este código. ¿Qué está mal?

```
public class LibroDAOImpl implements LibroDAO {
    public LibroDTO buscarPorIsbn(String isbn) {
        // conexión a la base
        // ...
        return new LibroDTO(...);
    }
}
```

UTN SANTA FE

La respuesta

El DAO está devolviendo un DTO directamente.

Está mezclando las dos responsabilidades que tanto trabajo nos dio separar.

¿Por qué duele? El mismo libro necesita mostrarse **distinto** en tres pantallas:

Una con precio	Otra sin precio	Otra solo con el título para una lista

Si el DAO devuelve un DTO fijo → estás atado. Si el DAO devuelve la entidad → armás el DTO que quieras en cada caso.

UTN SANTA FE

Data Transfer Object

Profundizando en esta faceta para armar diferentes DTOs de acuerdo a la pantalla que esté mostrando.



DTO es un patrón muy efectivo para transmitir información entre un cliente y un servidor. Permite crear estructuras de datos independientes de nuestro modelo de datos.



Creamos las “vistas” necesarias acorde a los clientes. Es decir, de un mismo conjunto de datos, podemos crear diferentes DTO que se ajusten a un mismo objeto de negocio.



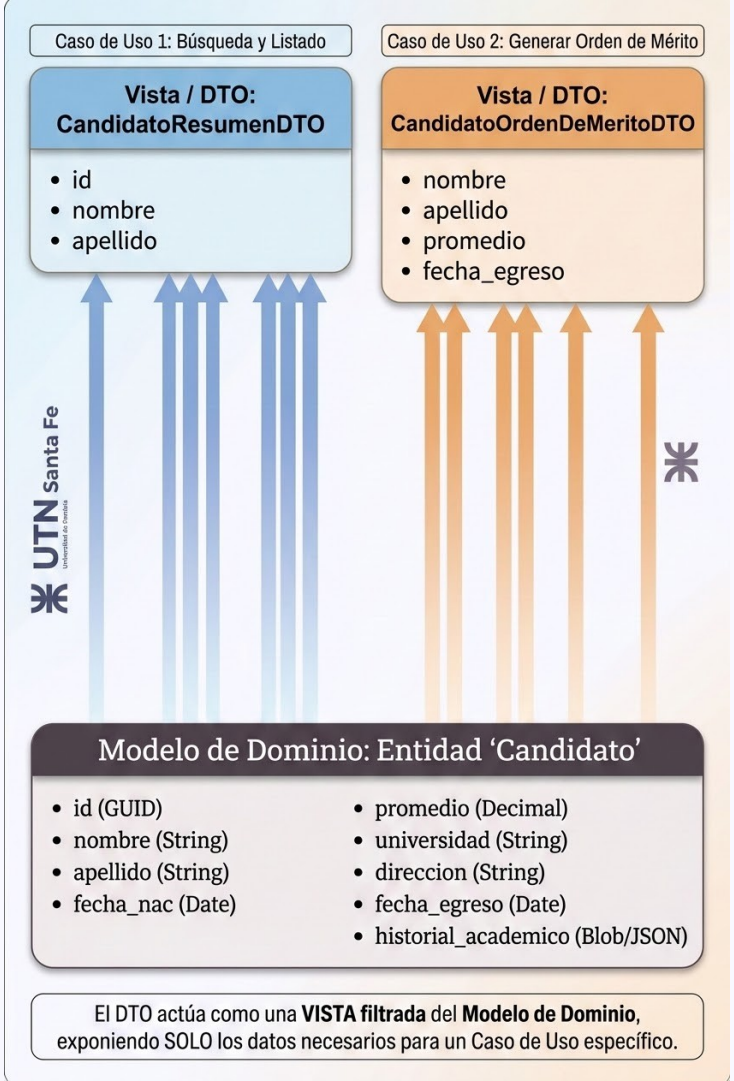
Permite controlar el formato, nombre y tipos de datos con los que transmitimos los datos para ajustarnos a los requerimientos.

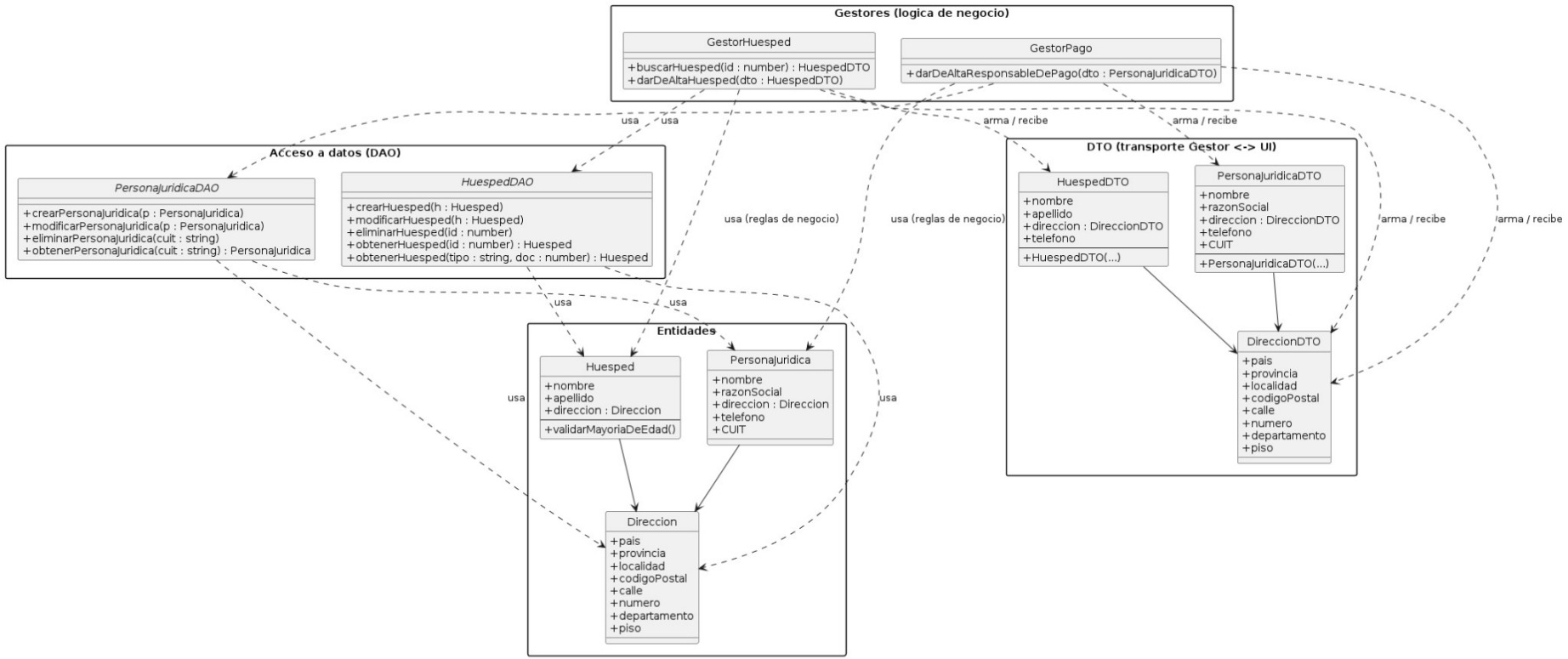


Si por alguna razón, el modelo cambia, el cliente no se ve afectado (bajo acoplamiento).

UTN SANTA FE

Ejemplo: Candidato




Gestores (logica de negocio)
GestorHuesped
+buscarHuesped(id : number) : HuespedDTO
+darDeAltaHuesped(dto : HuespedDTO)
GestorPago
+darDeAltaResponsableDePago(dto : PersonaJuridicaDTO)
usa
usa
arma / recibe
arma / recibe
Acceso a datos (DAO)
PersonaJuridicaDAO
+crearPersonaJuridica(p : PersonaJuridica)
+modificarPersonaJuridica(p : PersonaJuridica)
+eliminarPersonaJuridica(cuit : string)
+obtenerPersonaJuridica(cuit : string) : PersonaJuridica
HuespedDAO
+crearHuesped(h : Huesped)
+modificarHuesped(h : Huesped)
+eliminarHuesped(id : number)
+obtenerHuesped(id : number) : Huesped
+obtenerHuesped(tipo : string, doc : number) : Huesped
usa (reglas de negocio)
usa (reglas de negocio)
DTO (transporte Gestor <-> UI)
HuespedDTO
+nombre
+apellido
+direccion : DireccionDTO
+telefono
+HuespedDTO(...)
PersonaJuridicaDTO
+nombre
+razonSocial
+direccion : DireccionDTO
+telefono
+CUIT
+PersonaJuridicaDTO(...)
arma / recibe
arma / recibe
DireccionDTO
+pais
+provincia
+localidad
+codigoPostal
+calle
+numero
+departamento
+piso
usa
usa
Entidades
Huesped
+nombre
+apellido
+direccion : Direccion
+validarMayoriaDeEdad()
PersonaJuridica
+nombre
+razonSocial
+direccion : Direccion
+telefono
+CUIT
usa
usa
Direccion
+pais
+provincia
+localidad
+codigoPostal
+calle
+numero
+departamento
+piso

Implementación del patrón DAO

1. Identificamos las clases de dominio → Business Objects
2. Implementar una clase DTO por cada clase de dominio a utilizar → el DTO solo tiene los atributos necesarios que quiero almacenar/acceder en la base de datos → NO va a tener métodos con lógica de negocio
3. Definimos una interfaz para cada DAO
    a. Una interfaz DAO por cada clase de dominio con las operaciones CRUD (Create, Read, Update, Delete) y los métodos extra que necesitemos
4. Implementamos la clase que proporciona el código para la funcionalidad de cada interfaz

UTN SANTA FE

Ejercicio Práctico

Formen grupos de 3.

Recuperado del enunciado: el sistema importa candidatos desde otro sistema (CU006) con estos datos: número de candidato, apellido, nombre, tipo y número de documento, fecha de nacimiento, género, nacionalidad y email.

Además, cuando un consultor evalúa candidatos (CU025), el sistema genera automáticamente un token de 8 caracteres para que cada candidato entre a completar su cuestionario (CU026).

Candidato

- nroCandidato
- apellido
- nombre
- tipoDocumento
- nroDocumento
- fechaNacimiento
- genero
- nacionalidad
- email
- token

UTN SANTA FE

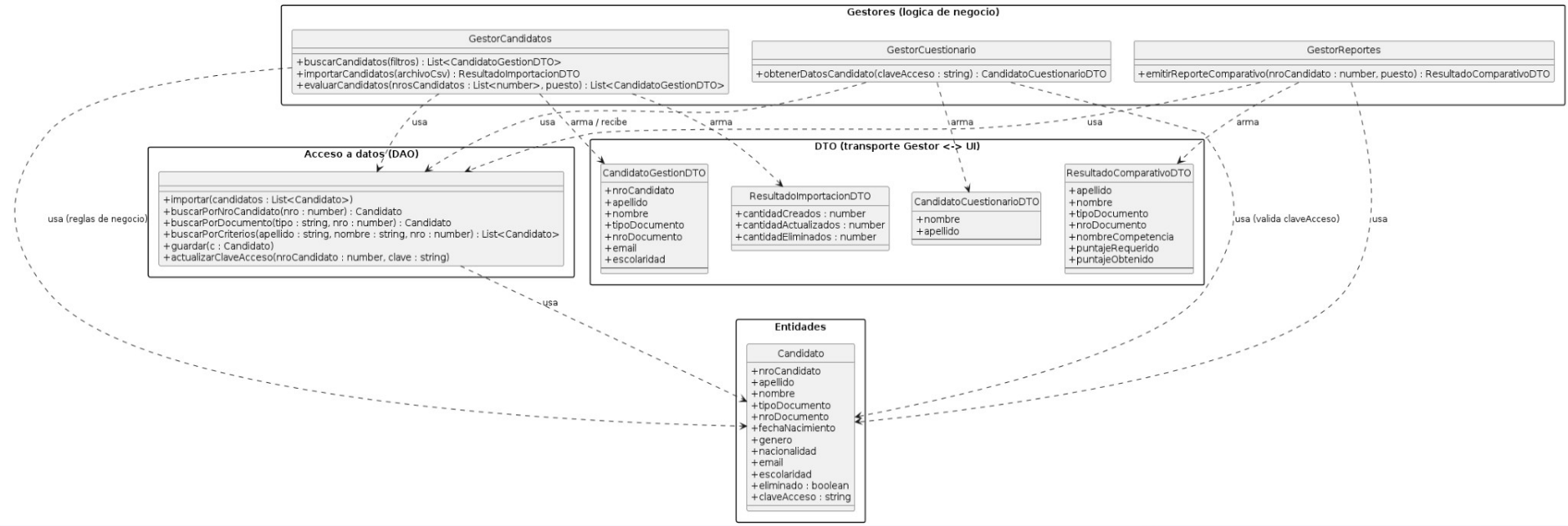
Ejercicio Práctico

1 - Diseñen la interfaz CandidatoDAO con al menos 4 métodos, pensando en qué necesitan CU006 (Importar Candidatos), CU025 (Evaluar a Candidato) y CU026 (Completar Cuestionario).

2 - Diseñen dos DTOs distintos:

- CandidatoGestionDTO: para la pantalla que usa el Consultor cuando busca y administra candidatos (CU002, CU025).
- CandidatoCuestionarioDTO: para lo que ve el propio Candidato una vez autenticado, al entrar a completar su cuestionario (CU026).

En este caso, veran que pueden implementar más de dos DTOs, pero vamos a quedarnos con los dos pedidos


Gestores (logica de negocio)
GestorCandidatos
+buscarCandidatos(filtros) : List<CandidatoGestionDTO>
+importarCandidatos(archivoCsv) : ResultadoImportacionDTO
+evaluarCandidatos(nrosCandidatos : List<number>, puesto) : List<CandidatoGestionDTO>
GestorCuestionario
+obtenerDatosCandidato(claveAcceso : string) : CandidatoCuestionarioDTO
GestorReportes
+emitirReporteComparativo(nroCandidato : number, puesto) : ResultadoComparativoDTO
usa
usa
arma / recibe
arma
arma
usa
arma
Acceso a datos (DAO)
+importar(candidatos : List<Candidato>)
+buscarPorNroCandidato(nro : number) : Candidato
+buscarPorDocumento(tipo : string, nro : number) : Candidato
+buscarPorCriterios(apellido : string, nombre : string, nro : number) : List<Candidato>
+guardar(c : Candidato)
+actualizarClaveAcceso(nroCandidato : number, clave : string)
DTO (transporte Gestor <-> UI)
CandidatoGestionDTO
+nroCandidato
+apellido
+nombre
+tipoDocumento
+nroDocumento
+email
+escolaridad
ResultadoImportacionDTO
+cantidadCreados : number
+cantidadActualizados : number
+cantidadEliminados : number
CandidatoCuestionarioDTO
+nombre
+apellido
ResultadoComparativoDTO
+apellido
+nombre
+tipoDocumento
+nroDocumento
+nombreCompetencia
+puntajeRequerido
+puntajeObtenido
usa (reglas de negocio)
usa (valida claveAcceso)
usa
usa
Entidades
Candidato
+nroCandidato
+apellido
+nombre
+tipoDocumento
+nroDocumento
+fechaNacimiento
+genero
+nacionalidad
+email
+escolaridad
+eliminado : boolean
+claveAcceso : string

UTN SANTA FE

TP - Etapas programadas

Próxima entrega

- **14/08/2026 - ETAPA 3 - Diagrama de clases ajustado**

https://blog.planttext.com/2023/04/05/class-diagrams/

Todos los puntos que requieran la confección de diagramas (2, 3, 4, 5, 6) deberán realizarse utilizando la herramienta https://www.planttext.com/. Donde se entregará en un archivo comprimido el código utilizado para confeccionar el diagrama y el diagrama final.



CONSULTAS??